

CSE 565 Performance Testing Project

Chase Coleman

Arizona State University

CSE 565: Software Verification and Validation

Performance Testing Project

October 4, 2026

1 Introduction

This report evaluates the behavior of a representative multi-task workload as the requested load increases from 100 to 1,000 and 10,000 logical users. The original instructor-provided script, `load_test_tasks.py`, was preserved without modification. A separate benchmark runner was developed to execute the same categories of work while collecting reproducible timing, throughput, and failure measurements.

2 Test Environment and Workload

2.1 Environment

The experiments were executed on an ARM64 Apple Silicon macOS 26.6.2 computer using Python 3.14.6 on October 4, 2026. The standard benchmark profile, simulated API mode, and a maximum of 100 active worker threads were used for every scenario. The runner schedules additional logical users as workers become available. The simulated API mode intentionally avoids dependence on public-network conditions; therefore, the reported results characterize the controlled local workload rather than an external service.

2.2 Benchmark Design

The separate runner, `performance_benchmark.py`, models each logical user with six sequential tasks: data processing, a simulated database delay, an API request, file input/output, computation, and logging. It limits active worker threads with `--max-workers`; consequently, a 10,000-user run represents 10,000 requested logical users without requiring 10,000 operating-system threads at once. Each user receives a distinct temporary file, and logging is synchronized to avoid the shared-file race in the starter code.

The controlled simulated API mode is used by default. This avoids sending thousands of unrequested requests to a public endpoint and makes the run more reproducible. If real API behavior is part of the experiment, identify the endpoint, timeout, date, and network conditions, and clearly distinguish those results from controlled simulated results.

2.3 Execution Procedure

The following command runs all required logical-user levels and produces one timestamped folder per scenario under `results/`. The default standard profile should be used consistently across the three compared runs.

```
python3 performance_benchmark.py --all --profile standard --max-workers 100
```

Each result folder contains `summary.csv`, which records sample count, failures, mean, median, 95th percentile, 99th percentile, minimum, maximum, overall elapsed time, and throughput for each task. The completed result folders were preserved as evidence. The values in Tables 1 and 2 were transcribed directly from those files.

3 Results

Table 1: Overall results by logical-user level.

Users	Elapsed (s)	Throughput (users/s)	Completed	Failures
100	0.118	847.210	100	0
1,000	0.957	1,044.978	1,000	0
10,000	9.421	1,061.440	10,000	0

Table 2: Task latency summary from `summary.csv`.

Users	Task	Mean (ms)	Median (ms)	P95 (ms)
100	Data processing	0.732	0.724	0.792
100	Database query	12.643	12.683	15.043
100	API request	18.158	18.242	20.637
100	File I/O	6.013	7.056	13.316
100	Computation	0.039	0.038	0.045
100	Logging	13.930	10.159	37.276
1,000	Data processing	0.758	0.758	0.790
1,000	Database query	12.209	12.431	13.336
1,000	API request	17.708	17.760	19.470
1,000	File I/O	2.458	2.038	6.569
1,000	Computation	0.039	0.038	0.042
1,000	Logging	55.760	59.859	62.657
10,000	Data processing	0.764	0.763	0.788
10,000	Database query	12.138	12.255	13.230
10,000	API request	17.708	17.727	19.409
10,000	File I/O	2.212	2.041	3.030
10,000	Computation	0.038	0.038	0.042
10,000	Logging	60.663	60.574	66.198

All task categories produced one successful sample per logical user. The file I/O mean at 100 users is higher than at the larger levels because startup and short-run contention have a greater influence on a small sample; it should not be interpreted as evidence that disk performance improves under arbitrary load.

4 Analysis

The overall elapsed time scaled close to linearly after the smallest scenario. Increasing the workload from 1,000 to 10,000 users increased elapsed time from 0.957 to 9.421 seconds, a factor of 9.84 for a tenfold increase in logical users. Throughput increased from 847.210 users per second at 100 users to 1,061.440 users per second at 10,000 users. The smaller scenario includes thread-pool and file-system startup costs that are amortized at the larger levels. The 1,000- and 10,000-user rates are close, indicating that this controlled workload approached a stable local throughput near 1,050 users per second under the selected 100-worker limit.

The synchronized logging task was the principal latency constraint. Its mean latency increased from 13.930 ms at 100 users to 55.760 ms at 1,000 users and 60.663 ms at 10,000 users. Its P95 was also the highest of all task categories at each scenario: 37.276 ms, 62.657 ms, and 66.198 ms, respectively. The lock serializes writes to protect the shared log, so this increase is expected as more workers contend for the same resource. In contrast, data processing, computation, the simulated database delay, and the simulated API delay were stable across the larger scenarios. The controlled API task had a 10,000-user P95 of 19.409 ms, whereas logging reached 66.198 ms.

No task failures were recorded at any load level. The maximum-worker setting causes users beyond the first 100 to wait before a worker begins processing them. The per-task timings in Table 2 measure execution time after a worker starts; they do not include that initial queue wait. Consequently, overall elapsed time and throughput are the appropriate measures for the full logical-user scenarios, while the task figures identify which executed operations became slower under contention.

5 Limitations and Recommendations

The benchmark represents logical users on one local machine; it is not a distributed production load test. The standard profile uses a controlled API response by default, so it measures the application’s workload pattern rather than a third-party service’s capacity. The 100-worker cap also intentionally models bounded concurrency rather than 10,000 simultaneously active threads. For future work, use a dedicated test endpoint, execute repeated trials, graph confidence intervals or variation across trials, measure end-to-end queue waiting explicitly, and consider a distributed load-testing tool when the goal is to generate true concurrent network traffic at very high scale.

6 Conclusion

The controlled benchmark completed all 11,100 requested logical-user executions without recorded failures. Overall throughput stabilized near 1,050 users per second for the 1,000- and 10,000-user scenarios, while elapsed time increased nearly in proportion to the load. The shared logging operation was the limiting component because its lock-protected writes produced the largest mean and P95 latencies at higher loads. A practical next improvement is to replace the single synchronized log with buffered or asynchronous logging, then repeat the same three scenarios to determine whether the resulting reduction in contention improves sustained throughput.

References

- Python Software Foundation. (n.d.). *concurrent.futures* — *Launching parallel tasks*. Python documentation. Retrieved October 4, 2026, from <https://docs.python.org/3/library/concurrent.futures.html>
- Python Software Foundation. (n.d.). *time* — *Time access and conversions*. Python documentation. Retrieved October 4, 2026, from <https://docs.python.org/3/library/time.html>